

生成 AI 利用申告書

- 氏名：塩澤 匠生
- 学籍番号：25G1065
- プログラムファイル名：HCK02_03.ino / HCK02_03.pde
- 使用した AI ツール：Claude Code (Opus 4.7)

1. 何に使ったか

発展課題は、提出課題 2 の Processing 波形描画に FFT を加えてスペクトルアナライザにする課題である。授業（ハッカソン 1 第 2 回）ではマイク波形を Serial Plotter / Processing に表示するところまでで、FFT（高速フーリエ変換）の扱い方は出てこなかった。Arduino 側（HCK02_03.ino）は提出課題 2 の HCK02_02.ino と完全に同じコードを流用しており、新たに書く必要があるのは Processing 側（HCK02_03.pde）のみだった。そこで、提出課題 2 で書いた HCK02_02.pde に FFT スペクトル表示を追加した HCK02_03.pde の生成を生成 AI に依頼した。Minim の FFT クラスの使い方や、Arduino 側のサンプリング周期と FFT の周波数軸（ナイキスト周波数）の対応の取り方も合わせてコードに含めてもらった。

2. AI への指示（プロンプト）

提出課題2 で書いた HCK02_02.pde（マイクの波形を Processing のウィンドウに描画するプログラム）をベースに、下段に FFT で周波数ごとの強さをバーグラフで表示するスペクトルアナライザに拡張したいです。

- Arduino 側のコードは HCK02_02.ino をそのまま使うので、
1 行 1 サンプル ADC 値（0-1023）が送られてきます
- delayMicroseconds(100) なのでサンプリング周波数は約 5 kHz の想定
- 楽曲再生（Minim）の部分はそのまま残してください
- 上段に時間波形、下段に FFT 結果のバー、という 2 段構成にしてください

Minim の FFT クラスを使った、いちばんシンプルな書き方で書いてほしいです。

3. AI の出力の使い方

- ☒ そのまま使った
- ☐ 一部修正して使った
- ☐ 参考にしただけ

→ 上で選んだ理由：

受け取った HCK02_03.pde は、提出課題 2 の HCK02_02.pde に対して

- new FFT(FFT_SIZE, SAMPLE_RATE) で FFT インスタンスを作る
- 入力は [-1, +1] に正規化した float 配列を用意し、serialEvent のたびに 1 サンプルずつ更新する
- draw() で fft.forward(...) を呼んだあと、fft.specSize() 個のビンを fft.getBand(i) で取り出して棒の高さに使う
- 横軸の Hz 表示は map(hz, 0, SAMPLE_RATE/2, 0, width) で位置を計算する

が上に乗ったかたちになっており、提出課題 2 で書いた波形描画と楽曲再生の部分はそのまま残っていた。動かしてみると上段に時間波形・下段にスペクトルバーがきれいに描かれ、要件をそのまま満たしていたので、

このコードをそのまま組み込んだ。

新たに加わった FFT 部分については、提出課題 2 のコードとの差分を取るかたちで 1 行ずつ読み直し、

- なぜ入力を $[-1, +1]$ に正規化するのか（中心電圧ぶんを引いて直流成分を抜くため）
- なぜ $\text{SAMPLE_RATE} = 5000.0$ を Arduino 側の `delayMicroseconds(100)` と対応させる必要があるのか（FFT の周波数軸が実際の Hz と一致するため）
- ナイキスト周波数 ($\text{SAMPLE_RATE} / 2 = 2.5 \text{ kHz}$) までは意味のあるバーは出ないこと

を自分の言葉で説明できるところまで確認したうえで提出した。

4. 正しいと確認した方法

- Arduino IDE で `HCK02_03.ino` がコンパイルでき、提出課題 2 と同じくシリアルモニタに 1 行 1 サンプルで流れることを確認した
- Processing IDE で `HCK02_03.pde` を起動し、上段に時間波形・下段にスペクトルバーが描画されることを確認した
- キー p で楽曲を再生してマイクに入力したとき、楽曲の音 (C4 / G4 / A4) 付近のバーが立ち上がる様子を観察した
- キー 1（正弦波）で再生したときは基音のバー 1 本がはっきり立ち、3（のこぎり波）や 4（矩形波）では倍音にあたる位置にもバーが並ぶことを確認した。これにより波形の違いと FFT 結果の対応が理解できているか確かめた
- Arduino 側 `delayMicroseconds(100)` と Processing 側 $\text{SAMPLE_RATE} = 5000.0$ が対応しており、ナイキスト周波数が 2.5 kHz になることをコード上で確認した

5. 問題や間違い

- ☐ あった
- ☒ なかった

AI が出してきた `HCK02_03.pde` は、提出課題 2 のコードに FFT 表示を素直に足したシンプルな構成で、窓関数や dB 変換など余分な処理は入っていなかった。そのまま動かして要件を満たしていたので、特に手を入れる必要はなかった。なお起動直後はバッファがゼロのままなので最初の数フレームはバーがすべて 0 になるが、これはサンプルが貯まるまでの待ち時間であり異常ではないと理解できた。

6. 自分で工夫した点

特になし。Arduino 側は提出課題 2 の `HCK02_02.ino` をそのまま流用し、Processing 側も AI が出してきたコードをそのまま採用した。

参考文献

- 有本泰子, 佐波孝彦「ハッカソン 1 第 2 回」講義資料 (`HCK1_02.pdf`, `HCK1_02.ex.pdf`)
- 生成 AI 利用申告書テンプレート（ハッカソン 1 講義資料）
- Processing Reference: `Serial`, `bufferUntil`, `serialEvent`
<https://processing.org/reference/libraries/serial/>

- Minim Reference: FFT, AudioOutput, Oscil, Line
<https://code.compartmental.net/minim/>
- Minim FFT Javadoc:
<https://code.compartmental.net/minim/javadoc/ddf/minim/analysis/FFT.html>